Guardsix Fleet Device Broker

A standalone, zero-dependency Python tool for **RHEL 9** (stdlib only) that interacts with the Guardsix Fleet API to:

1. **List devices** across SIEM pools and nodes.
2. **Safely bulk-delete devices** with automatic **DeviceGroup membership cleanup**.

No external packages (`requests`, `PyYAML`, `python-dotenv`, etc.) are required — everything runs on the Python standard library shipped with RHEL 9 (Python 3.9+).

Table of Contents

Features

- **Zero dependencies** — uses only `urllib`, `json`, `csv`, `ssl`, `logging`, `argparse`, and `pathlib`.
- **Credential isolation** — API token and base URL are read from a local `.env` file; they are never logged or printed.
- **JSON-based topology** — SIEM pools and nodes are declared in a simple `pools.json` file.
- **Async job monitoring** — every delete operation is monitored until the Director reports `success`, `completed`, or an explicit `failed` status with a reason.
- **DeviceGroup synchronisation** — when a device is successfully deleted, it is automatically removed from all DeviceGroups it belonged to. If the delete fails, **no** DeviceGroup is modified.
- **Localhost protection** — any device named `localhost` or carrying IP `127.0.0.1` / `::1` is **never** eligible for deletion.
- **Dry-run mode** — simulate a bulk deletion before touching the Director.
- **Full logging** — every API call, retry, and result is written to a timestamped log file under `logs/`.

Prerequisites

- RHEL 9 (or any Linux distribution with Python **3.9+**).
- Network access to the Guardsix Fleet API.
- A valid **API Bearer token** with permissions to read Devices / DeviceGroups and delete Devices.

Installation

1. Copy the entire `device_broker/` directory to the target server (e.g. via `scp`, `rsync`, or a USB stick).
2. No `pip install` is required.

```
cd lp_tenant_importer_v2/device_broker
cp .env.sample .env
cp pools.json.sample pools.json
```

3. Edit `.env` and `pools.json` (see Configuration).

Configuration

.env file

Create a `.env` file in the same directory as the script:

```
LP_DIRECTOR_URL=https://director.company.com
LP_DIRECTOR_API_TOKEN=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

Variable	Description
`LP_DIRECTOR_URL`	Base URL of the Guardsix Fleet Director (no trailing slash).
`LP_DIRECTOR_API_TOKEN`	Your Bearer token.

> **Security note:** Keep file permissions strict: `chmod 600 .env`.

pools.json file

Define the SIEM pools and nodes you want to manage:

```
{
  "pools": [
    {
      "name": "production",
      "pool_uuid": "2bc98ce1-6a6d-4c4c-82e3-62bc11691488",
      "nodes": [
        {
          "name": "search_head_01",
          "node_id": "19fc6388-0b01-8d97-8aab-e9162b851a9d"
        }
```

```
        ]
      }
    ]
}
```

Field	Description
`name`	Human-readable pool name (used only in logs).
`pool_uuid`	The Director pool UUID.
`nodes`	List of SIEM nodes belonging to this pool.
`nodes[].name`	Human-readable node name.
`nodes[].node_id`	The Director node UUID.

Workflows

1. List devices

Generate an export file containing every device discovered in the configured pools:

```
python -m lp_tenant_importer_v2.device_broker list
```

Or, if you run from inside the `device_broker` folder:

```
python __main__.py list
```

Output: `devices_export.txt` (tab-separated values).

Example content:

```
pool_uuid   node_id device_id   device_name ip  device_groups   delete
2bc98ce1-...    19fc6388-...    abc123  firewall-01 10.0.0.1    dg1,dg2
##no##
2bc98ce1-...    19fc6388-...    def456  localhost   127.0.0.1       ##no##
```

- The `delete` column defaults to `##no##` for **all** devices.
- **Do not** change the header row.
- Use `sed` to bulk-switch every line:
  `sed -i 's/##no##/##yes##/g' devices_export.txt`
- Then edit the file and revert `##yes##` back to `##no##` for any device you want to **keep**.

> **Tip:** You can specify a custom output file with `-o`:

```
python -m lp_tenant_importer_v2.device_broker list -o
/tmp/my_export.txt
```

---

2. Bulk delete devices

After editing the export file, run the bulk deletion:

```
python -m lp_tenant_importer_v2.device_broker bulk-delete
```

The tool will:

1. Read `devices_export.txt`.
2. Keep only rows where `delete=##yes##`.
3. For each pool/node group:
   - Fetch **live** DeviceGroups from the Director.
   - Delete each device and **monitor the async job**.
   - If deletion succeeds, remove the device from every DeviceGroup it belonged to.
   - If deletion fails, **skip** the DeviceGroup update for that device and continue with the next one.

Dry-run first (highly recommended):

```
python -m lp_tenant_importer_v2.device_broker bulk-delete --dry-run
```

> **Note:** Even if you accidentally set `delete=##yes##` for a localhost device, the tool will **forcibly skip** it and log an explicit error.

Use a custom input file:

```
python -m lp_tenant_importer_v2.device_broker bulk-delete -i
/tmp/my_export.txt
```

Safety mechanisms

Mechanism	Behaviour
Manual approval file	Deletions are driven by a human-edited TSV file. If `delete` normalised value is not exactly `yes`, the device is ignored.
Localhost guard	Devices named `localhost` or with IP `127.0.0.1` / `::1` are automatically skipped, even if `delete=##yes##`.

Mechanism	Behaviour
DG sync on success only	DeviceGroups are updated **only** when the Director confirms the device deletion succeeded. If the delete fails, the DG memberships are left untouched.
SSL verification	Enabled by default. Use `--no-verify-ssl` only in lab environments.
No hard deletions	The tool only calls the Director `DELETE` endpoint; there is no filesystem deletion.

Logging

Every execution creates a timestamped log file under `logs/`:

```
logs/device_broker_20260526_094733.log
```

- **Console** output is `INFO` level and above.
- **File** output is `DEBUG` level and above (full request/response bodies, except the Bearer token which is redacted).

Example log excerpt:

```
2026-05-26 09:47:33,377 | INFO     | device_broker | list_devices |
Retrieved 42 devices from pool=2bc98... node=19fc6...
2026-05-26 09:47:33,378 | INFO     | device_broker | delete_device |
Deleting device abc123 from pool=2bc98... node=19fc6...
2026-05-26 09:47:35,123 | INFO     | device_broker | monitor_job | Job
completed successfully
2026-05-26 09:47:35,124 | INFO     | device_broker | update_device_group |
Removed device abc123 from DG dg1 (dg-id-1)
```

Command reference

Global flags

Flag	Default	Description
`--env-file`	`.env`	Path to the API credentials file.
`--pools-file`	`pools.json`	Path to the topology file.
`--log-dir`	`logs`	Directory for timestamped log files.
`--no-verify-ssl`	`False`	Disable SSL certificate verification.

`list`

Flag	Default	Description
`-o`, `--output`	`devices_export.txt`	Path to the TSV export file.

`bulk-delete`

Flag	Default	Description
`-i`, `--input`	`devices_export.txt`	Path to the TSV file to read.
`--dry-run`	`False`	Print what would be deleted without calling the API.

Troubleshooting

`Configuration error: Environment file not found: .env`

The tool cannot find your `.env` file. Ensure it exists in the working directory or pass an explicit path:

```
python -m lp_tenant_importer_v2.device_broker --env-file /path/to/.env list
```

`HTTP 401 on GET ...`

The API token is invalid or expired. Verify `LP_DIRECTOR_API_TOKEN` in your `.env` file.

`HTTP 404 on DELETE ...`

The device may have already been deleted manually, or the `device_id` in the TSV file is stale. The tool logs the failure and continues with the next device.

`SSL: CERTIFICATE_VERIFY_FAILED`

Your RHEL 9 server does not trust the Director's HTTPS certificate. In a lab environment you can bypass verification with `--no-verify-ssl`. In production, import the CA certificate into the system trust store.

`No devices marked for deletion`

The `delete` column in your TSV file does not contain any `yes` values. Remember the check is case-insensitive (`yes`, `YES`, and `Yes` all work).

FAQ

Q: Can I run this on a non-RHEL system?
A: Yes, any system with Python 3.9+ will work. The tool was designed specifically for RHEL 9 environments where `pip install` is prohibited.

Q: Why JSON instead of YAML for `pools.json`?
A: JSON is part of the Python standard library. YAML would require `PyYAML`, which is not available without `pip`.

Q: Does the tool support proxy servers?
A: The stdlib `urllib` honours the standard `http_proxy` / `https_proxy` environment variables. Set them before running the script:

```
export https_proxy=http://proxy.company.com:8080
python -m lp_tenant_importer_v2.device_broker list
```

Q: What happens if the Director returns an async job for a delete?
A: The tool polls the `monitorapi` endpoint every 2 seconds for up to 120 seconds. It interprets `response.success`, `response.errors`, and `status` fields exactly like DirSync v2.

Q: Can I accidentally delete the localhost device?
A: No. The localhost guard is hardcoded at two levels:

1. During `list`, localhost devices are exported with `delete=##no##`.
2. During `bulk-delete`, the tool performs an explicit pre-scan of every row marked `##yes##`. Any device matching localhost criteria is **forcibly skipped** with an `ERROR` log, regardless of what the file says.

Q: Is there a way to undo a bulk delete?
A: No. Deletions are permanent on the Director. Always use `--dry-run` first and keep a backup of the last known good `devices_export.txt`.

License

Internal use only — maintained for the ESA/Nexova DirSync ecosystem.